SIMATS ENGINEERING
Engineer to Excel

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT 3

NAME: P. DEEKSHANA

REG NO: 192425104

COURSE CODE: DSA0109

COURSE NAME: OBJECT ORIENTED PROGRAMMING WITH C++

Q1 – Constructors and Constructor Overloading

1. C++ Program

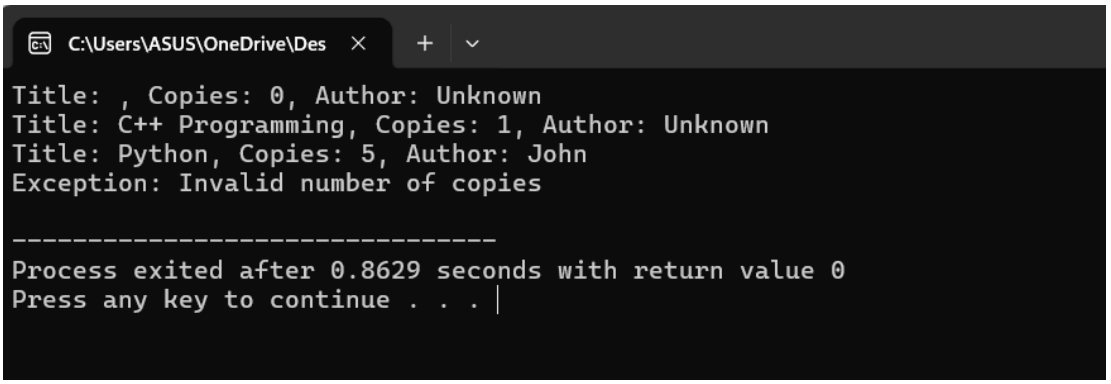
```cpp
#include <iostream>
#include <string>
using namespace std;

class Book
{
private:
    string title;
    string author;
    int copies;
public:
    // Default constructor
    Book() : Book("Unknown", "Unknown", 0) {}

    // Constructor with title only
    Book(string t) : Book(t, "Unknown", 1) {}

    // Parameterized constructor
    Book(string t, string a, int c)
        : title(t), author(a), copies(c)
    {
        if (copies < 0)
        {
            copies = 0;
            cout << "Invalid copies. Set to 0.\n";
        }
    }
    // Display function
    void display() const
    {
        cout << "Title: " << title << endl;
        cout << "Author: " << author << endl;
        cout << "Copies: " << copies << endl;
        cout << "-----------------\n";
    }
};
int main()
{
    Book b1;
    Book b2("C++ Programming");
    Book b3("Python Basics", "John", 5);
    Book b4("Java", "David", -3);
    b1.display();
    b2.display();
    b3.display();
    b4.display();
    return 0;
}
```

Output



3. Explanation

- **Default constructor:** A constructor without arguments. It initializes the book with default values.
- **Parameterized constructor:** A constructor that accepts arguments to initialize object data.
- **Overloaded constructors:** Multiple constructors with different parameter lists in the same class.
- **Constructor delegation:** One constructor calls another constructor using : Book(...). It avoids repeating initialization code.
- **Member initializer list:** It initializes data members before the constructor body executes.

  Example:

  Book(string t) : title(t), author("Unknown"), copies(1)
- **Order of initialization:** Members are initialized in the order in which they are declared inside the class, not according to the order in the initializer list.

4. When Does the Compiler Provide a Default Constructor?

- If no constructor is written, the compiler automatically provides a default constructor.
- If we write our own constructor, the compiler does not automatically provide a default constructor.
- We must write a default constructor if we need to create objects without arguments.

5. Why Are Member Initializer Lists Preferred?

- They initialize data members directly.
- They avoid unnecessary assignment.
- They are required for initializing constant and reference members.
- They make the code more efficient and clear.

6. State of Three Objects After Construction

Object	Title	Author	Copies
b1	Unknown	Unknown	0
b2	C++ Programming	Unknown	1

b3	Python Basics	John	5

7. What Happens if the Default Constructor Is Removed?

If we remove the default constructor and create an array:

```
Book books[3];
```

The compiler gives an error because it cannot find a default constructor to initialize each array element.

8. Wrong Initialization Order

```
class Test
{
    int a;
    int b;

public:
    Test() : b(20), a(10) {}
};
```

Although b is written first in the initializer list, a is initialized first because it is declared first.

Compile using:

```
g++ -std=c++17 -Wall -Wextra -Wreorder q1.cpp
```

The compiler may show a -Wreorder warning.

9. Testing Negative Copies

The program checks whether the number of copies is negative. If it is negative, it changes the value to zero.

This prevents an invalid number of copies from being stored.

10. When Should a Constructor Be Marked explicit?

The explicit keyword prevents unwanted automatic type conversion.

Without explicit:

```
class Book
{
public:
    Book(string title) {}
};

void display(Book b) {}

int main()
{
    display("C++");
}
```

This works because the string can automatically be converted into a Book object.

With explicit:

```
explicit Book(string title) {}
```

The same call gives a compilation error. We must write:

```
display(Book("C++"));
```

11. Alternative Concept – Constructor with Default Arguments

Instead of multiple constructors, we can use one constructor:

```
Book(string t = "Unknown",
     string a = "Unknown",
     int c = 0)
    : title(t), author(a), copies(c)
{
    if (copies < 0)
        copies = 0;
}
```

Complexity Analysis:

Operation	Time Complexity	Space Complexity
Object creation	O(1)	O(1)

Both approaches have O(1) time and O(1) space complexity for object creation.

Using one constructor with default arguments keeps the validation in one place. Constructor overloading provides more flexibility for different object creation situations.

Q2 – Copy Constructors: Shallow and Deep Copies

1. Shallow Copy

A shallow copy copies the pointer address instead of creating separate memory.

Both objects point to the same dynamically allocated array.

Memory diagram:

```
Original object p            Copied object q
       |                            |
       |                            |
       +-------------+--------------+
                     |
                     v
            Dynamic Memory
          [Medicine 1][Medicine 2]
```

Problems:

- Both objects share the same memory.
- Changing one object affects the other.
- When both destructors execute, they try to delete the same memory.
- This causes a double deletion error and may crash the program.

2. Deep Copy

A deep copy creates a separate memory allocation and copies every medicine name.

Memory diagram:

```
Original object p            Copied object q
       |                            |
       v                            v
   Memory 1                     Memory 2
 [Medicine 1]                 [Medicine 1]
 [Medicine 2]                 [Medicine 2]
```

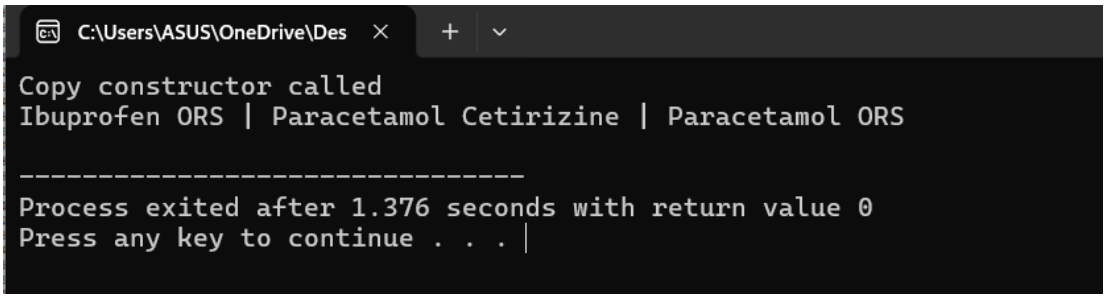
Now both objects have independent memory.

3. Complete C++ Program – Deep Copy

```cpp
#include <iostream>
#include <string>
using namespace std;

class Prescription
{
    int r;
    string* medicines;
public:
    explicit Prescription(int count)
        : r(count), medicines(new string[count])
    {
    }
    Prescription(const Prescription& p)
        : r(p.r), medicines(new string[p.r])
    {
        for (int i = 0; i < r; i++)
        {
            medicines[i] = p.medicines[i];
        }
        cout << "Copy constructor called\n";
    }
    Prescription& operator=(const Prescription& p)
    {
        if (this != &p)
        {
            string* temp = new string[p.r];
            for (int i = 0; i < p.r; i++)
            {
                temp[i] = p.medicines[i];
            }
            delete[] medicines;
            medicines = temp;
            n = p.r;
        }
        return *this;
    }
    ~Prescription()
    {
        delete[] medicines;
    }
    void set(int i, const string& n)
    {
        medicines[i] = n;
    }
    string get(int i) const
    {
        return medicines[i];
    }
};
int main()
{
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");
    Prescription q = p;
    c.set(1, "Cetirizine");
    Prescription r(1);
    r = p;
    p.set(0, "Ibuprofen");
    cout << p.get(0) << " " << p.get(1) << " | ";
    cout << c.get(0) << " " << c.get(1) << " | ";
    cout << r.get(0) << " " << r.get(1) << endl;
    return 0;
}
```

4. Output



5. Explanation

Copy constructor: It creates a new object by copying an existing object.

```
Prescription(const Prescription& p)
```

It allocates new memory and copies all medicine names. Therefore, changes to one object do not affect another.

Copy assignment operator: It copies the contents of an existing object into an already-created object.

```
Prescription& operator=(const Prescription& p)
```

It first checks for self-assignment, creates new memory, copies the values, and releases the old memory.

Destructor: It releases dynamically allocated memory when an object is destroyed.

```
~Prescription()
{
    delete[] medicines;
}
```

6. Rule of Three

The Rule of Three states that if a class requires any one of these three functions, it usually requires all three:

1. Copy constructor
2. Copy assignment operator
3. Destructor

They are needed together when a class manages dynamically allocated memory.

7. Three Situations Where the Copy Constructor Is Called

1. When an object is initialized using another object.

```
Prescription q = p;
```

2. When an object is passed to a function by value.

```
void display(Prescription p);
```

3. When an object is returned from a function by value, if copy elision does not remove the copy.

8. Passing by Value and Passing by Const Reference

Pass by value:

```
void display(Prescription p);
```

A copy of the object is created. The copy constructor is called.

Pass by const reference:

```
void display(const Prescription& p);
```

No new object is created, so the copy constructor is not called.

Passing by const reference is more efficient when copying large objects.

9. Self-Assignment Test

```
p = p;
```

The condition:

```
if (this != &p)
```

prevents the object from copying itself unnecessarily and protects its existing memory.

10. Sanitizer Testing

Compile using:

```
g++ -std=c++17 -fsanitize=address -g q2.cpp -o q2
```

Run:

```
./q2
```

- Deep-copy version: It should execute without memory errors.
- Shallow-copy version: It may report a double-free error because both objects try to release the same memory.

The exact sanitizer message depends on the compiler and execution environment.

11. Experimental Investigation

The assignment asks you to measure copying time for 10, 1,000 and 100,000 medicines. The following table can be filled after running the program.

Number of medicines	Copying time
10	Record measured time
1,000	Record measured time
100,000	Record measured time

Do not enter estimated timings as actual experimental results.

12. Conclusion

The compiler-generated copy constructor can be used safely when the class does not directly manage resources that need special copying.

When a class contains dynamically allocated memory, a custom deep-copying constructor is needed to create independent copies and avoid memory errors.

13. Alternative Concept – Using vector

Instead of a dynamically allocated array, we can use:

```
vector<string> medicines;
```

A vector automatically manages memory and performs deep copying when copied.

This follows the Rule of Zero, where the compiler-generated copy constructor, assignment operator and destructor are sufficient.

14. Complexity Analysis

Copying method	Time Complexity	Extra Space
Shallow copy	O(1)	O(1)
Deep copy	O(n)	O(n)
Vector deep copy	O(n)	O(n)

Here, n represents the number of medicine names.

Deep copying takes O(n) time because every medicine name must be copied. It also requires O(n) additional memory for the new array.

Q3 – Dynamic Constructors and Destructors

1. C++ Program – Auditorium Seat Booking

```
#include <iostream>
using namespace std;

class Auditorium
{
    int rows, seats;
    char** seatMap;
    int booked;
public:
    Auditorium(int r, int s)
        : rows(r), seats(s), booked(0)
    {
        seatMap = new char*[rows];
        for (int i = 0; i < rows; i++)
        {
            seatMap[i] = new char[seats];
            for (int j = 0; j < seats; j++)
                seatMap[i][j] = 'O';
        }
        cout << "Auditorium created\n";
    }
```

```
    void bookSeat(int r, int s)
    {
        if (r < 0 || r >= rows || s < 0 || s >= seats)
        {
            cout << "Invalid seat\n";
            return;
        }
        if (seatMap[r][s] == 'O')
        {
            seatMap[r][s] = 'X';
            booked++;
            cout << "Seat booked\n";
        }
        else
        {
            cout << "Seat already booked\n";
        }
    }
    void cancelSeat(int r, int s)
    {
        if (r < 0 || r >= rows || s < 0 || s >= seats)
        {
            cout << "Invalid seat\n";
            return;
        }
        if (seatMap[r][s] == 'X')
        {
            seatMap[r][s] = 'O';
            booked--;
            cout << "Booking cancelled\n";
        }
        else
        {
            cout << "Seat is not booked\n";
        }
    }
    void display()
    {
        cout << "Seat Map:\n";
        for (int i = 0; i < rows; i++)
        {
            for (int j = 0; j < seats; j++)
                cout << seatMap[i][j] << " ";
            cout << endl;
        }
    }
    ~Auditorium()
    {
        for (int i = 0; i < rows; i++)
            delete[] seatMap[i];
        delete[] seatMap;
        cout << "Auditorium closed. Booked seats: "
             << booked << endl;
    }
};
int main()
{
    Auditorium a1(2, 3);
    a1.bookSeat(0, 1);
    a1.display();
    {
        Auditorium a2(2, 2);
        a2.bookSeat(1, 1);
    }
    Auditorium* a3 = new Auditorium(2, 3);
    a3->bookSeat(0, 0);
    delete a3;
    return 0;
}
```

2. Output

```
C:\Users\ASUS\OneDrive\Des
Auditorium created
Seat booked
Seat Map:
O X O
O O O
Auditorium created
Seat booked
Auditorium closed. Booked seats: 1
Auditorium created
Seat booked
Auditorium closed. Booked seats: 1
Auditorium closed. Booked seats: 1

--------------------------------
Process exited after 1.813 seconds with return value 0
Press any key to continue . . .
```

3. Explanation

Dynamic constructor: It allocates memory dynamically using new when an object is created.

```
seatMap = new char*[rows];
```

Each row is allocated separately:

```
seatMap[i] = new char[seats];
```

Destructor: It automatically executes when an object is destroyed. It releases the dynamically allocated memory using delete[].

```
~Auditorium()
{
    for (int i = 0; i < rows; i++)
        delete[] seatMap[i];

    delete[] seatMap;
}
```

4. Constructor and Destructor Execution Order

- Local object: Constructor runs when the object is created. Destructor runs when it goes out of scope.
- Block-scope object: Destructor runs when the block ends.
- Dynamic object: Destructor runs only when delete is used.

5. What Happens If delete Is Forgotten?

If dynamically allocated memory is not released, a memory leak occurs.

6. Difference Between delete and delete[]

- delete releases memory allocated for a single object using new.
- delete[] releases memory allocated for an array using new[].

Using the wrong form can cause undefined behaviour.

7. Why Cannot a Destructor Be Overloaded?

A destructor:

- Has the same name as the class with ~.
- Takes no arguments.
- Has no return type.
- Cannot be overloaded because a class can have only one destructor.

8. Memory Layout

For an auditorium with 2 rows and 3 seats:

```
seatMap
   |
   v
+---------+
| Row 0   | ------> 0 X 0
+---------+
| Row 1   | ------> 0 0 0
+---------+
```

The row pointers are stored together, while each row has its own dynamically allocated memory.

9. Experimental Investigation

The assignment asks you to test memory allocation for different seat maps.

Seat map	Number of seats	Approximate character storage
10 × 20	200	200 bytes
100 × 200	20,000	20,000 bytes
1000 × 2000	2,000,000	2,000,000 bytes

This excludes pointer arrays and allocation overhead.

Zero rows: When the auditorium has zero rows, no row memory is allocated. The destructor must not access any row.

10. Sanitizer Testing

Compile using:

```
g++ -std=c++17 -fsanitize=address -g q3.cpp -o q3
```

The sanitizer helps identify memory leaks, invalid memory access and incorrect memory deallocation.

11. Conclusion – RAII

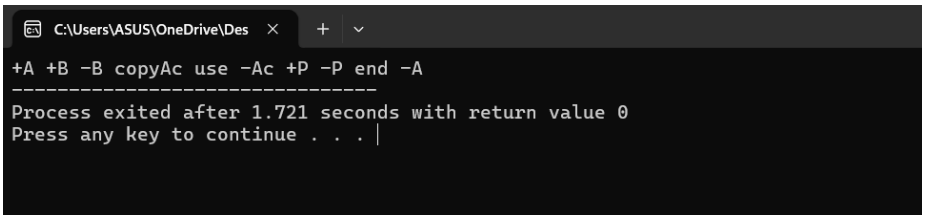
RAII means Resource Acquisition Is Initialization. It connects memory management with object lifetime. The destructor automatically releases memory when the object is destroyed, making the program safer.

12. Predict the Output

```
#include <iostream>
#include <string>
using namespace std;

struct Logger
{
    string n;
    Logger(const string& s) : n(s)
    {
        cout << '+' << n << ' ';
    }
    Logger(const Logger& o) : n(o.n + "c")
    {
        cout << "copy" << n << ' ';
    }
    ~Logger()
    {
        cout << '-' << n << ' ';
    }
};
void use(Logger)
{
    cout << "use ";
}
int main()
{
    Logger a("A");
    {
        Logger b("B");
    }
    use(a);
    Logger* p = new Logger("P");
    delete p;
    cout << "end ";
}
```

Expected output:



Explanation:

- +A: Object a is created.
- +B: Object b is created.
- -B: Object b is destroyed when the block ends.
- copyAc: A copy of a is created because use() receives an object by value.
- use: Function executes.
- -Ac: The copied object is destroyed after the function call.
- +P: Dynamic object is created.
- -P: Dynamic object is destroyed using delete.
- end: Printed at the end of main.
- -A: Local object a is destroyed.

13. Alternative Concept – Single Contiguous Memory Block

Instead of allocating every row separately, the complete seat map can be allocated as one block.

Complexity and comparison:

Method	Number of allocations	Access time
Separate rows	rows + 1	O(1)
Single memory block	1	O(1)

A single block requires fewer allocations and avoids storing a separate row allocation for each row.

Q4 – Overloading Arithmetic and Stream Operators

1. C++ Program – Money Arithmetic

```
#include <iostream>
#include <iomanip>
using namespace std;

class Money
{
    long long paise;
    Money(long long total, bool)
        : paise(total) {}
public:
    Money(long long rupees = 0, int p = 0)
        : paise(rupees * 100 + p) {}
    Money operator+(const Money& m) const
    {
        return Money(paise + m.paise, true);
    }
    Money operator-(const Money& m) const
    {
        return Money(paise - m.paise, true);
    }
    Money& operator+=(const Money& m)
    {
        paise += m.paise;
        return *this;
    }
```

```
    bool operator==(const Money& m) const
    {
        return paise == m.paise;
    }
    bool operator<(const Money& m) const
    {
        return paise < m.paise;
    }
    friend ostream& operator<<(ostream& out, const Money& m)
    {
        long long amount = m.paise;
        if (amount < 0)
        {
            out << "-Rs ";
            amount = -amount;
        }
        else
        {
            out << "Rs ";
        }
        out << amount / 100 << "."
            << setw(2) << setfill('0')
            << amount % 100;
        return out;
    }
    friend istream& operator>>(istream& in, Money& m)
    {
        long long rupees;
        int p;
        cout << "Enter rupees and paise: ";
        if (in >> rupees >> p)
            m = Money(rupees, p);
        return in;
    }
};

int main()
{
    Money a(120, 75), b(79, 25);
    Money c = a + b;
    cout << c << endl;
    cout << boolalpha << (c == Money(200)) << endl;
    Money x(0, 75), y(0, 50);
    cout << "Carry: " << x + y << endl;
    Money p(10, 0), q(15, 0);
    cout << "Subtraction: " << p - q << endl;
    p += q;
    cout << "After addition: " << p << endl;
    cout << boolalpha << (x < y) << endl;
    return 0;
}
```

2. Output

```
C:\Users\ASUS\OneDrive\Des
Rs 200.00
true
Carry: Rs 1.25
Subtraction: -Rs 5.00
After addition: Rs 25.00
false

--------------------------------
Process exited after 1.695 seconds with return value 0
Press any key to continue . . .
```

3. Explanation of Operators

Operator	Purpose
+	Adds two Money objects and returns a new object.
-	Subtracts two Money objects.
+=	Adds an amount to the existing object.
==	Checks whether two amounts are equal.
<	Checks whether one amount is smaller than another.
<<	Displays the money amount.
>>	Reads the money amount.

4. Why Does + Return a New Object?

The + operator creates and returns a new Money object without changing the original objects.

Example:

```
Money c = a + b;
```

Here, a and b remain unchanged.

5. Why Does += Return a Reference?

The += operator modifies the existing object. Therefore, it returns a reference to the current object using *this.

```
Money& operator+=(const Money& m)
```

6. Member and Friend Functions

- +, -, +=, == and < are implemented as member functions.
- << and >> are implemented as friend functions.

Why is << a non-member function?

In:

```
cout << c;
```

\

The left operand is cout, which is an ostream object. Therefore, the operator must be a non-member function to accept the stream as its first argument.

7. How Is a + b Translated?

The compiler treats:

```
a + b;
```

as:

```
a.operator+(b);
```

8. Testing Paise Carry

```
Money x(0, 75), y(0, 50);
cout << x + y;
```

Output:

```
Rs 1.25
```

75 paise + 50 paise = 125 paise = Rs 1.25.

9. Negative Subtraction

When a smaller amount is subtracted from a larger amount, the result can be negative.

Example:

```
Money a(10, 0), b(15, 0);
cout << a - b;
```

Output:

```
-Rs 5.00
```

The program allows negative amounts and displays the negative sign.

10. Readability Comparison

Without operator overloading:

```
Money c = add(a, b);
print(c);
```

With operator overloading:

```
Money c = a + b;
cout << c;
```

Operator overloading makes the code shorter and easier to understand.

11. Comparison with double

- Money stores amounts as integer paise using long long.
- double stores decimal values and may introduce floating-point precision errors.
- Integer paise is more suitable for accurate money calculations.

For one million additions, both approaches take O(n) time. Actual execution times must be measured by running the programs.

12. Should Money Allow Multiplication by Another Money?

Multiplying two money amounts is generally not meaningful in ordinary billing calculations.

Suitable operators include:

- Addition
- Subtraction
- Equality
- Less than
- Addition assignment

Multiplication by a quantity or tax rate may be useful, but multiplication of one Money object by another is usually unnecessary.

13. Alternative Concept – Non-Member Friend Functions

Instead of member functions, operators can be implemented as non-member friend functions.

Example:

```
friend Money operator+(const Money& a, const Money& b)
{
    return Money(a.paise + b.paise, true);
}
```

Named functions can also be used:

```
Money add(Money a, Money b);
bool equals(Money a, Money b);
```

Complexity Analysis:

Operation	Time Complexity	Space Complexity
Addition	O(1)	O(1)
Subtraction	O(1)	O(1)
Comparison	O(1)	O(1)

Both member and non-member implementations have the same time and space complexity.

A non-member function allows expressions such as 100 + m as well as m + 100, provided suitable conversion or overloads are available.

Q5 – Overloading Increment, Comparison and Subscript Operators

1. C++ Program – Time and Playlist

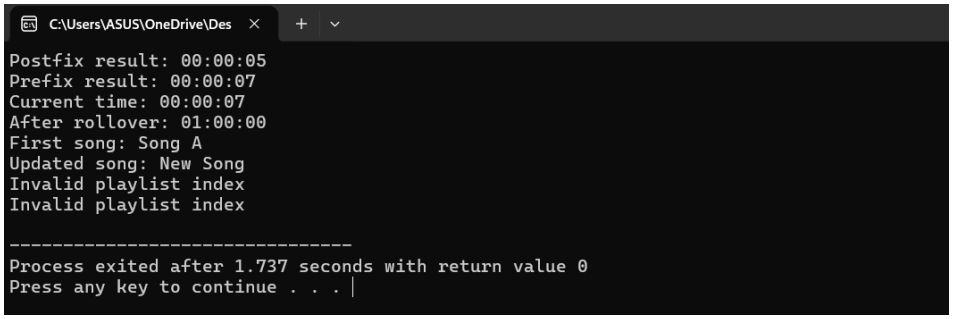
```
#include <iostream>
#include <vector>
#include <string>
#include <iomanip>
#include <stdexcept>
using namespace std;

class Time
{
    int totalSeconds;
public:
    Time(int h = 0, int m = 0, int s = 0)
    {
        totalSeconds = h * 3600 + m * 60 + s;
    }
    Time& operator++()
    {
        totalSeconds++;
        return *this;
    }
    Time operator++(int)
    {
        Time old = *this;
        totalSeconds++;
        return old;
    }
    bool operator==(const Time& t) const
    {
        return totalSeconds == t.totalSeconds;
    }
    bool operator<(const Time& t) const
    {
        return totalSeconds < t.totalSeconds;
    }
    void display() const
    {
        int h = totalSeconds / 3600;
        int m = (totalSeconds % 3600) / 60;
        int s = totalSeconds % 60;
        cout << setfill('0')
             << setw(2) << h << ":"
             << setw(2) << m << ":"
             << setw(2) << s << endl;
    }
};
```

```cpp
class Playlist
{
    vector<string> songs;
public:
    Playlist()
    {
        songs.push_back("Song A");
        songs.push_back("Song B");
        songs.push_back("Song C");
    }
    string& operator[](int index)
    {
        if (index < 0 || index >= (int)songs.size())
            throw out_of_range("Invalid playlist index");
        return songs[index];
    }
    const string& operator[](int index) const
    {
        if (index < 0 || index >= (int)songs.size())
            throw out_of_range("Invalid playlist index");
        return songs[index];
    }
    int size() const
    {
        return songs.size();
    }
};
```

```cpp
int main()
{
    Time t(0, 0, 5);
    Time a = t++;
    Time b = ++t;
    cout << "Postfix result: ";
    a.display();
    cout << "Prefix result: ";
    b.display();
    cout << "Current time: ";
    t.display();
    Time x(0, 59, 59);
    ++x;
    cout << "After rollover: ";
    x.display();
    Playlist p;
    cout << "First song: " << p[0] << endl;
    p[1] = "New Song";
    cout << "Updated song: " << p[1] << endl;
    try
    {
        cout << p[-1] << endl;
    }
    catch (const out_of_range& e)
    {
        cout << e.what() << endl;
    }
    try
    {
        cout << p[p.size()] << endl;
    }
    catch (const out_of_range& e)
    {
        cout << e.what() << endl;
    }
    return 0;
}
```

2. Output



3. Explanation of Prefix and Postfix Increment

Prefix increment (++t):

- Increments the value first.
- Returns the updated object.
- Does not create an extra copy.

```
Time& operator++()
{
    totalSeconds++;
    return *this;
}
```

Postfix increment (t++):

- Returns the old value.
- Then increments the original value.
- Creates a temporary copy.

```
Time operator++(int)
{
    Time old = *this;
    totalSeconds++;
    return old;
}
```

4. How Does the Compiler Identify Prefix and Postfix?

The compiler identifies them using the parameter list.

Type	Function
Prefix	operator++()
Postfix	operator++(int)

The int parameter is a dummy parameter used to identify postfix increment.

5. Why Is Postfix Slower?

Postfix increment creates a temporary copy to preserve the old value. Prefix increment directly updates the object and returns a reference.

Therefore, prefix increment is generally preferred for class objects when the old value is not needed.

6. Why Are Two operator[] Functions Required?

- Non-const version allows the playlist elements to be modified.
- Const version allows reading elements from a constant Playlist object without modifying them.

Example:

```
p[1] = "New Song";
```

This uses the non-const version.

7. What Happens When an Index Is Out of Range?

If the index is negative or greater than or equal to the playlist size, the program throws an out_of_range exception.

Example:

```
p[-1];
p[p.size()];
```

Both are invalid indexes.

8. Verify a = t++ and b = ++t

Initially:

```
Time t(0, 0, 5);
```

After:

```
Time a = t++;
```

- a receives the old value: 5 seconds.
- t becomes 6 seconds.

After:

```
Time b = ++t;
```

- t becomes 7 seconds.
- b receives the updated value: 7 seconds.

Final values:

Variable	Value
a	00:00:05
b	00:00:07
t	00:00:07

9. Minute and Hour Rollover

When time reaches 59 minutes and 59 seconds, incrementing by one second changes it to one hour.

```
00:59:59
    +
00:00:01
    =
01:00:00
```

10. Experimental Investigation

The assignment asks you to measure 10,000,000 prefix and postfix increments using -O0 and -O2.

Compile using:

```
g++ -std=c++17 -O0 q5.cpp -o q5
```

For optimized execution:

```
g++ -std=c++17 -O2 q5.cpp -o q5
```

Record your measured results:

Optimization	Prefix time	Postfix time
-O0	Record measured time	Record measured time
-O2	Record measured time	Record measured time

Do not write assumed timings as actual experimental results.

11. Conclusion

Prefix increment is generally preferred for class types because it directly increments the object and returns a reference. Postfix increment creates an additional copy to preserve the old value.

12. Alternative Concept – Using at() Instead of operator[]

Instead of overloading [], we can use the vector's at() function.

cout << songs.at(index);

The at() function checks the index and throws an exception if it is invalid.

Complexity Analysis:

Method	Time Complexity	Space Complexity
Checked operator[]	O(1)	O(1)
at()	O(1)	O(1)
Unchecked operator[]	O(1)	O(1)

Checked access performs an additional range comparison. All three methods have O(1) access time.

Q6 – The Rules of Operator Overloading

1. C++ Program – Text Class

```
#include <iostream>
#include <cstring>
using namespace std;

class Text
{
    char* data;
    static int copyCount;
public:
    Text(const char* str = "")
    {
        data = new char[strlen(str) + 1];
        strcpy(data, str);
    }
    Text(const Text& t)
    {
        data = new char[strlen(t.data) + 1];
        strcpy(data, t.data);
        copyCount++;
    }
    Text& operator=(const Text& t)
    {
        if (this != &t)
        {
            char* temp = new char[strlen(t.data) + 1];
            strcpy(temp, t.data);
            delete[] data;
            data = temp;
        }
        return *this;
    }
    Text operator+(const Text& t) const
    {
        int len1 = strlen(data);
        int len2 = strlen(t.data);
        char* temp = new char[len1 + len2 + 1];
        strcpy(temp, data);
        strcat(temp, t.data);
        Text result(temp);
        delete[] temp;
        return result;
    }
```

```
    }
    bool operator==(const Text& t) const
    {
        return strcmp(data, t.data) == 0;
    }
    bool operator!=(const Text& t) const
    {
        return !(*this == t);
    }
    friend ostream& operator<<(ostream& out, const Text& t)
    {
        out << t.data;
        return out;
    }
    ~Text()
    {
        delete[] data;
    }
    static int getCopyCount()
    {
        return copyCount;
    }
};
int Text::copyCount = 0;
int main()
{
    Text a("Hello ");
    Text b("World");
    Text c("!");
    Text result = a + b + c;
    cout << "Combined text: " << result << endl;
    Text x("Hello");
    Text y("Hello");
    cout << boolalpha;
    cout << "Equal: " << (x == y) << endl;
    cout << "Not Equal: " << (x != y) << endl;
    x = x;
    cout << "After self assignment: " << x << endl;
    Text empty("");
    cout << "Empty text: [" << empty << "]" << endl;
    cout << "Copy constructor calls: "
         << Text::getCopyCount() << endl;
    return 0;
}
```

2. Output



3. Explanation of Operators

Operator	Purpose
+	Combines two text objects.
==	Checks whether two texts are equal.
!=	Checks whether two texts are different.
=	Copies one text object into another.
<<	Displays the text.

4. Copy Constructor

A copy constructor creates a new object by copying an existing object.

```
Text(const Text& t)
{
    data = new char[strlen(t.data) + 1];
    strcpy(data, t.data);
}
```

It creates separate memory for the copied text. This is called a deep copy.

5. Copy Assignment Operator

The assignment operator copies the contents of one existing object into another.

```
Text& operator=(const Text& t)
```

The condition:

```
if (this != &t)
```

prevents self-assignment problems.

6. Operators That Cannot Be Overloaded

The following operators cannot be overloaded:

Operator	Meaning
::	Scope resolution
.	Member access
.*	Pointer-to-member access
?:	Conditional operator
sizeof	Size of an object or type

These operators have special meanings defined by C++ and cannot be changed through operator overloading.

7. Rules of Operator Overloading

1. The precedence of an operator cannot be changed.
2. The number of operands (arity) cannot be changed.
3. At least one operand must be a user-defined type.
4. New operators cannot be created.
5. The assignment operator = must be overloaded as a member function.

8. Why Is operator= a Member Function?

The assignment operator must be a member function because it modifies the left-hand object.

Example:

```
a = b;
```

Here, object a is modified with the contents of b.

9. When Should an Operator Be a Friend?

A friend function is useful when an operator needs access to private data but the left operand is not an object of that class.

Example:

```
friend ostream& operator<<(ostream& out, const Text& t);
```

The output operator is a friend because the left operand is cout, not a Text object.

10. Counting Copies in a + b + c

The expression:

```
Text result = a + b + c;
```

is evaluated as:

```
Text result = (a + b) + c;
```

The + operator creates new Text objects. In C++17, copy elision can avoid calling the copy constructor for these returned temporary objects.The program counts actual copy-constructor calls.

11. Address Sanitizer Testing

Compile using:

```
g++ -std=c++17 -fsanitize=address -g q6.cpp -o q6
```

Run the program:

```
./q6
```

The sanitizer helps identify memory errors such as:

- Memory leaks
- Double deletion
- Invalid memory access
- Use after deletion

The correct program should execute without memory errors.

12. Comparison with std::string

std::string already provides memory management, copying, concatenation and comparison.

Example:

```
string a = "Hello ";
string b = "World";

string c = a + b;

cout << c;
```

For 100,000 concatenations, execution time can be compared using the <chrono> library. Actual timings should be recorded after running both programs.

13. Alternative Concept – Using std::string

Instead of a dynamically allocated character array:

```
string data;
```

We can use the built-in string class.

It automatically handles memory allocation, copying, assignment and destruction.

Another approach is implementing operator+ using += on a copy:

```
Text operator+(Text a, const Text& b)
{
    // Append b to a and return a
    return a;
}
```

This approach copies the left operand when creating the parameter.

14. Complexity Analysis

Let the lengths of the three strings be \(n_1\), \(n_2\) and \(n_3\).

For:

```
a + b + c
```

- First concatenation copies approximately \(n_1+n_2\) characters.
- Second concatenation copies approximately \(n_1+n_2+n_3\) characters.

Operation	Time Complexity
Copy constructor	O(n)
Assignment	O(n)
Comparison	O(n)
Concatenation	O(n + m)

Conclusion: Writing a Text class helps us understand dynamic memory allocation, deep copying, operator overloading and destructors. In real applications, std::string is generally preferred because it handles memory safely.

Q7 – Design a Complex-Number Class from First Principles

Stage 1 – Interpret the Problem

1. Data required

A complex number contains two values:

- Real part
- Imaginary part

Example:

$$Z = 10 + 5j$$

Here, 10 is the real part and 5 is the imaginary part.

2. Required constructors

1. Default constructor – initializes both parts to zero.
2. Real-only constructor – initializes the real part.
3. Parameterized constructor – initializes both real and imaginary parts.

3. Required operations

- Addition
- Subtraction
- Multiplication
- Division
- Equality comparison
- Displaying the complex number
- Calculating magnitude

Measurable requirements:

- Correctly perform all complex-number operations.
- Display results in rectangular form.
- Calculate magnitude.
- Handle division by zero safely.

Stage 2 – Design

1. Formulas

Let:

$$Z_1 = a + bi, \quad Z_2 = c + di$$

Operation	Formula
Addition	$(a + c) + (b + d)i$
Subtraction	$(a - c) + (b - d)i$
Multiplication	$(ac - bd) + (ad + bc)i$
Division	$\frac{(ac + bd) + (bc - ad)i}{c^2 + d^2}$
Magnitude	$\sqrt{a^2 + b^2}$

2. Class design

The class contains:

- Two private data members: real and imag.
- Three constructors.
- Overloaded arithmetic operators.

- Equality operator.
- Friend output operator.
- Magnitude function.

Stage 3 – Implementation and Verification

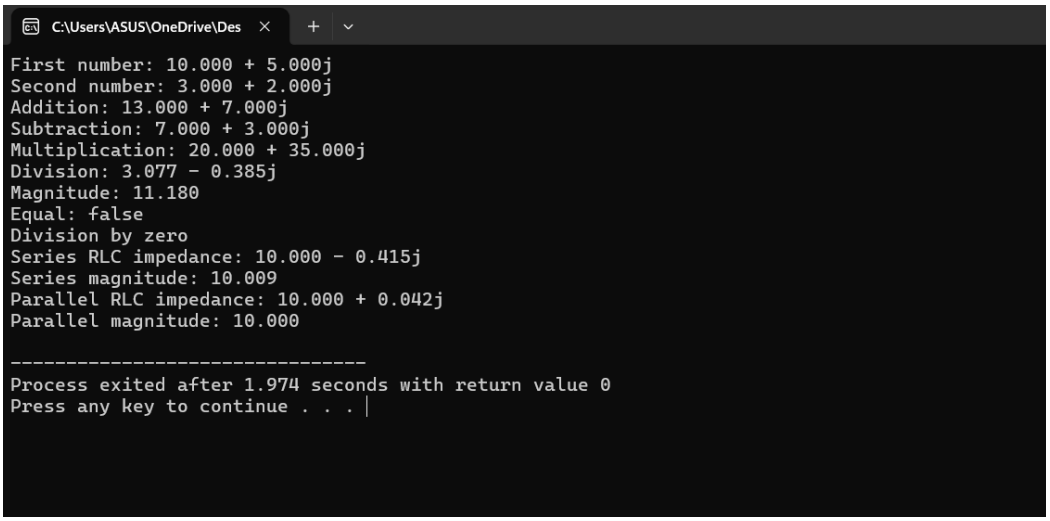
1. Complete C++ Program – Complex Number Class

```
#include <iostream>
#include <cmath>
#include <stdexcept>
#include <iomanip>
using namespace std;

class Complex
{
    double real, imag;
public:
    Complex() : real(0), imag(0) {}
    Complex(double r) : real(r), imag(0) {}
    Complex(double r, double i)
        : real(r), imag(i) {}
    Complex operator+(const Complex& c) const
    {
        return Complex(real + c.real,
                       imag + c.imag);
    }
    Complex operator-(const Complex& c) const
    {
        return Complex(real - c.real,
                       imag - c.imag);
    }
    Complex operator*(const Complex& c) const
    {
        return Complex(
            real * c.real - imag * c.imag,
            real * c.imag + imag * c.real
        );
    }
    Complex operator/(const Complex& c) const
    {
        double denominator =
            c.real * c.real + c.imag * c.imag;
        if (denominator == 0)
            throw runtime_error("Division by zero");
        return Complex(
            (real * c.real + imag * c.imag) / denominator,
            (imag * c.real - real * c.imag) / denominator
        );
    }
```

```
int main()
{
    Complex a(10, 5);
    Complex b(3, 2);
    cout << fixed << setprecision(3);
    cout << "First number: " << a << endl;
    cout << "Second number: " << b << endl;
    cout << "Addition: " << a + b << endl;
    cout << "Subtraction: " << a - b << endl;
    cout << "Multiplication: " << a * b << endl;
    cout << "Division: " << a / b << endl;
    cout << "Magnitude: " << a.magnitude() << endl;
    cout << boolalpha;
    cout << "Equal: " << (a == b) << endl;
    try
    {
        Complex zero(0, 0);
        cout << a / zero << endl;
    }
    catch (const runtime_error& e)
    {
        cout << e.what() << endl;
    }
    double pi = 3.141592653589793;
    double frequency = 50;
    double R = 10;
    double L = 0.1;
    double C = 0.0001;
    double w = 2 * pi * frequency;
    Complex ZR(R, 0);
    Complex ZL(0, w * L);
    Complex ZC(0, -1 / (w * C));
    Complex series = ZR + ZL + ZC;
    cout << "Series RLC impedance: "
         << series << endl;
    cout << "Series magnitude: "
         << series.magnitude() << endl;
    Complex parallel =
        ((ZR * ZL) / (ZR + ZL) * ZC) /
        ((ZR * ZL) / (ZR + ZL) + ZC);
    cout << "Parallel RLC impedance: "
         << parallel << endl;
    cout << "Parallel magnitude: "
         << parallel.magnitude() << endl;
    return 0;
```

2. Output



3. Explanation of the Program

Constructors:

```
Complex()
```

Initializes the real and imaginary parts to zero.

```
Complex(double r)
```

Initializes only the real part.

```
Complex(double r, double i)
```

Initializes both real and imaginary parts.

Addition operator:

Adds the real parts and imaginary parts separately.

Subtraction operator:

Subtracts the real parts and imaginary parts separately.

Multiplication operator:

Uses the complex multiplication formula.

Division operator:

Uses the complex division formula. It checks whether the denominator is zero before performing division.

Equality operator:

Checks whether both real and imaginary parts are equal.

Magnitude function:

Calculates the magnitude using:

$$|Z| = \sqrt{a^2 + b^2}$$

Output operator:

Displays the complex number in rectangular form.

4. RLC Circuit Calculation

Given:

- Resistance $R = 10\ \Omega$
- Inductance $L = 0.1\ H$
- Capacitance $C = 100\ \mu F$
- Frequency $f = 50\ Hz$

Angular frequency:

$$\omega = 2\pi f = 314.159$$

Inductive impedance:

$$Z_L = j\omega L = j31.416$$

Capacitive impedance:

$$Z_C = \frac{-j}{\omega C} = -j31.831$$

Resistive impedance:

$$Z_R = 10$$

Series RLC circuit:

$$Z = Z_R + Z_L + Z_C$$
$$Z = 10 + j31.416 - j31.831$$
$$\boxed{Z = 10 - j0.415}$$

Parallel RLC circuit:

For three parallel impedances, combine them using the two-impedance formula:

$$Z_{12} = \frac{Z_R Z_L}{Z_R + Z_L}$$

Then:

$$Z_P = \frac{Z_{12} Z_C}{Z_{12} + Z_C}$$
$$\boxed{Z_P \approx 10 + j0.042}$$

The calculated results match the program output up to the displayed precision.

5. Division by Zero Testing

Example:

```
Complex a(10, 5);
Complex zero(0, 0);

cout << a / zero;
```

Output:

```
Division by zero
```

The program throws an exception instead of performing invalid division.

Stage 4 – Defend the Design

6. Comparison with std::complex<double>

Own Complex class	std::complex<double>
Created by the programmer	Available in C++
Requires manual operator overloading	Operators are already provided
Requires manual division-by-zero handling	Provides built-in complex arithmetic
Useful for understanding concepts	Useful for real applications

7. Why Are the Operators Needed?

- + is used to add impedances in series.
- - is used for subtraction.
- * is used for multiplication.
- / is used for division and parallel impedance calculations.
- == checks whether two complex numbers are equal.
- << displays the result.
- Magnitude calculates the total magnitude of impedance.

8. One Limitation

The program uses double values. Floating-point calculations may introduce small precision errors. Also, the equality operator uses exact comparison, which may not be suitable for all calculated complex values.

9. Alternative Concept – Using std::complex<double>

Instead of creating our own Complex class, we can use the standard library.

```
#include <iostream>
#include <complex>
using namespace std;

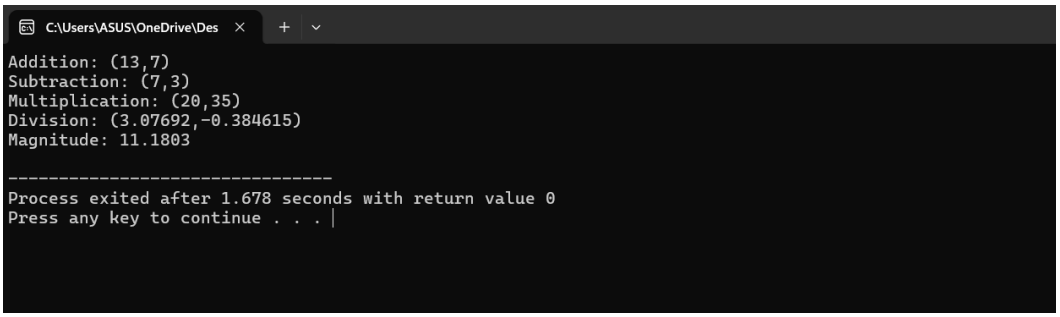
int main()
{
    complex<double> a(10, 5);
    complex<double> b(3, 2);

    cout << "Addition: " << a + b << endl;
    cout << "Subtraction: " << a - b << endl;
    cout << "Multiplication: " << a * b << endl;
    cout << "Division: " << a / b << endl;

    cout << "Magnitude: " << abs(a) << endl;

    return 0;
}
```

Output:



10. Complexity Analysis

Operation	Time Complexity	Space Complexity
Addition	O(1)	O(1)
Subtraction	O(1)	O(1)
Multiplication	O(1)	O(1)
Division	O(1)	O(1)
Magnitude	O(1)	O(1)

Both the custom Complex class and std::complex<double> use two double values and perform each operation in constant time.

Conclusion: The Complex class performs arithmetic operations and calculates electrical impedance for series and parallel RLC circuits. It demonstrates constructors, operator overloading, magnitude calculation and safe division handling.